



FOR JAVA DEVELOPERS MOVING TO NESTJS

1999 plus 1. "19991".

Spring Boot to NestJS, episode 33: gRPC

Same proto, different runtime types. The proto is shared; the language mapping is not.

One payments.proto with an int64 amount: Spring's handler got a long, so 1999 plus 1 was 2000.

Nest passes its loader options straight to proto-loader. With none, the amount was a Long object: 1999 plus 1 joined strings, and zero was not equal to 0.

A client deadline stopped the waiting on both stacks. The server still completed the charge on both.

In scope

- @GrpcMethod and the generated base class
- int64 and the loader options
- A throwing handler
- Deadlines and cancellation

Not in scope

- Streaming RPCs
- TLS and auth metadata

At the time of recording the Spring gRPC starters move into Spring Boot from 4.2.

01 The measurements

INT64 1999 + 1, SPRING	2000
INT64 1999 + 1, NEST DEFAULTS	"19991" (a Long object)
INT64, NEST longs: Number	2000
9007199254740993, longs: Number	arrived as ...992
A HANDLER THAT THROWS	UNKNOWN, masked on both
300 ms DEADLINE, 1 s CHARGE	charged anyway, both
CANCELLATION CHECKED FIRST	skipped, both

One proto, both servers, a grpc-js client.



02 Choose the int64 representation

Number is exact only through 2^{53} minus 1. String keeps every value and makes the arithmetic yours. Pick for the range your proto allows.

```
main.ts

NestFactory.createMicroservice<MicroserviceOptions>(AppModule, {
  transport: Transport.GRPC,
  options: {
    package: 'payments',
    protoPath: join(import.meta.dirname, 'payments.proto'),
    loader: { longs: Number, defaults: true },
  },
});
```

longs: Number: 1999 plus 1 was 2000; 9007199254740993 arrived as 9007199254740992

Run on a real machine, 2026-10-01, by scripts/verify-grpc.sh in the course repository.

03 Stop the work, not just the wait

Check for cancellation before the step with the side effect, and make that step idempotent: cancellation can race with completion.

```
payments.controller.ts

@GrpcMethod('Payments', 'Charge')
async charge(request: ChargeRequest, metadata: Metadata, call: ServerUnaryCall<ChargeRequest, ChargeReply>) {
  const quote = await this.prepare(request);
  if (call.cancelled) return;
  return this.ledger.chargeOnce(request.paymentId, quote);
}
```

cancellation checked before the side effect: completed 0, skipped 1 on both stacks

Run on a real machine, 2026-10-01, by scripts/verify-grpc.sh in the course repository.

Before you move a gRPC service to Nest

1. What does the loader turn int64 into?
2. Can any int64 in the proto exceed 2^{53} ?
3. Are zero values present, or missing, in the handler?
4. Does the handler stop work when the client gives up?
5. Is every side effect idempotent?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-grpc.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS gRPC https://docs.nestjs.com/microservices/grpc

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. gRPC is a trademark of The Linux Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.